Build a Website
with
Claude Code
HTML
CSS
Welcome!
claude init
Build
A Practical Guide for
Beginners and Builders

# Build a Website with Claude Code

*A practical guide for beginners and builders*

---

## Introduction

Here's a clean **introduction page** with an **About the Author**, written in your Impeccable.style tone 

---

# Introduction

## Why This Book Exists

Most people don't build websites because they think it's hard.

It used to be.

You had to:

- learn HTML
- learn CSS
- learn JavaScript
- understand frameworks
- deal with hosting

Now, that's optional.

What changed is simple:

> AI can write the code
> You define the outcome

This book is about that shift.

---

## What You'll Learn

Not theory. Not tutorials.

A process.

By the end, you will know how to:

- use the terminal without fear
- work with Claude Code effectively
- design clean, modern interfaces
- deploy your website in minutes
- build fast using an AI-first workflow

---

## Who This Is For

This is for you if:

- you want to build, not just learn
- you're tired of endless tutorials
- you want something live, fast

This is **not** for you if:

- you want to understand everything before starting
- you prefer theory over execution

---

## How to Use This Book

Don't read it like a novel.

Build while reading.

Each chapter is meant to be:
applied immediately

tested
improved

Speed matters more than perfection.

---

## The Only Mindset You Need

You are not here to become a developer.

You are here to become a **builder**.

You don't need to know everything.
You need to know how to:
ask
refine
ship

---

# About the Author

I'm François.

I don't believe in:

- long tutorials
- overcomplicated stacks
- learning without building

I believe in:
speed
clarity
execution

I use AI to:

- build faster
- remove friction
- turn ideas into real products

This book is not theory.

It’s the exact process I use.

---

## Why I Wrote This

Because I saw too many people stuck.

They:

- consume content
- take notes
- never ship

The gap is not skill.

It’s action.

So I wrote this to remove excuses.

---

## Final Note Before You Start

You don’t need more information.

You need to start.

Open your terminal.
Follow the steps.
Build something.

That’s where everything changes.

---

# Chapter 1 — Mastering the Terminal

## 1.1 What is the Terminal?

The terminal (also called *command line* or *shell*) is where you talk directly to your computer using text commands.

Instead of clicking buttons, you type instructions.

It might feel intimidating at first — but it's actually:

- faster
- more precise
- essential for developers

---

## 1.2 Why the Terminal Matters

When building a website, the terminal lets you:

- create and navigate projects
- install tools and dependencies
- run development servers
- deploy your website
- interact with AI coding tools like Claude Code

If you skip the terminal, you'll always be limited.

---

## 1.3 Opening the Terminal

- Press `Cmd + Space`
- Type `Terminal`
- Hit Enter

### On Linux:

- Press `Ctrl + Alt + T`

### On Windows:

- Use **Windows Terminal** or **PowerShell**

---

## 1.4 Essential Commands You Must Know

Think of these as your “survival kit”.

### ☑ Navigate folders

```
cd my-folder
```

Move into a folder

```
cd ..
```

Go back one level

---

### ☑ List files

```
ls
```

Shows what’s inside the current directory

---

### ☑ Create a folder

```
mkdir my-website
```

---

### ☑ Create a file

```
touch index.html
```

---

### ☑ Clear the terminal

```
clear
```

---

## 1.5 Understanding Your Current Location

```
pwd
```

This shows where you are in your system.

Example:

```
/Users/francois/projects/my-website
```

---

## 1.6 Installing Packages (Important)

Most modern web tools use package managers.

If you install Node.js, you'll use:

```
npm install
```

This downloads dependencies for your project.

---

## 1.7 Running a Project

To start a development server:

```
npm run dev
```

This launches your website locally (usually at `localhost:3000`).

---

## 1.8 Terminal Mindset

At first, you'll forget commands. That's normal.

What matters is:

- understanding what commands *do*
- not memorizing everything

With time, it becomes second nature.

---

# Chapter 2 — Install and Launch Claude Code

## 2.1 What is Claude Code?

Claude Code is an AI-powered development assistant that helps you:

- generate code
- fix bugs
- build full applications

understand complex systems

-

Instead of Googling everything, you *ask Claude*.

---

## 2.2 Prerequisites

Before installing Claude Code, make sure you have:

### Node.js installed

Check with:

```
node -v
```

If not installed, download it from:
https://nodejs.org

---

### A project folder

Create one:

```
mkdir my-ai-website
cd my-ai-website
```

---

## 2.3 Installing Claude Code

Depending on the setup (CLI or integration), installation typically looks like:

```
npm install -g @anthropic-ai/claude-code
```

This installs Claude Code globally on your machine.

---

## 2.4 Launching Claude Code

Once installed, you can start it with:

```
claude
```

Or inside a project:

```
claude init
```

This initializes Claude in your current folder.

---

## 2.5 First Interaction

Once Claude is running, you can type:

```
Build a simple landing page with HTML and CSS
```

Claude will:

- generate files
- structure your project
- explain what it’s doing

---

## 2.6 Example Workflow

Here’s a typical beginner flow:

```
mkdir my-site
cd my-site
claude init
```

Then:

 Ask Claude:

```
Create a modern homepage for a personal brand
```

Claude will:

- create HTML/CSS files

- suggest improvements
- guide you step-by-step

---

## 2.7 Running Your Website

After Claude generates your project:

```
npm install
npm run dev
```

Then open:
http://localhost:3000

---

## 2.8 The New Way of Building

Before:

- search tutorials
- copy/paste code
- debug for hours

Now:

- describe what you want
- iterate with AI
- build faster

Claude becomes your:

- developer partner
- mentor
- debugger

---

## End of Part 1

You now have:

terminal basics

Claude Code installed

your first AI-powered workflow

---

Absolutely — here's the revised section using **Claude Code as the interface**, not manual Git commands.

---

# Section — Initialize Git with Claude Code and Set Remote Origin

## Why This Matters

A website on your laptop is a draft.

A website in GitHub is:

- versioned
- backed up
- deployable

You want Claude to handle the setup, not do it piece by piece yourself.

---

## Use Claude as the Operator

Instead of manually typing every Git command, open Claude Code in your project folder and give it the task clearly.

Example:

```
claude
```

Then prompt:

```
Initialize a git repository for this project, create an initial
commit, and prepare it to connect to a GitHub remote.
```

Claude will inspect the folder, initialize Git, stage files, and create the first commit.

---

## Set the Remote Origin with Claude

Once you've created an empty repository on GitHub, copy its URL.

Then tell Claude:

```
Set the remote origin of this repository to: https://github.com/your-
username/my-website.git
```

Claude can configure the remote for you.

Then ask:

```
Verify that the remote origin is correctly set and that the default
branch is main.
```

This removes guesswork.

---

## Push the Project

Next prompt:

```
Push this project to GitHub and make sure the upstream branch is
configured correctly.
```

Claude will handle the remaining Git steps.

You stay focused on the outcome:

- project online
- repo clean
- workflow ready

---

## The Right Way to Prompt

Weak prompt:

```
Do git
```

Good prompt:

```
Initialize git for this project, create the first commit called
"initial commit", set the branch to main, connect the remote origin
to this GitHub repository, and push everything.
```

Precision matters.

Claude works better when the task is explicit.

---

## Recommended Workflow

Inside any new website project:

1. Launch Claude Code
2. Ask it to initialize Git
3. Ask it to connect the GitHub remote
4. Ask it to push the repository
5. Ask it to verify everything is clean

Example sequence:

```
Initialize git for this project and create the initial commit.
Set remote origin to https://github.com/your-username/my-website.git
Push the repository and configure upstream correctly.
Check that everything is connected properly and the working tree is
clean.
```

---

## Why This Is Better

You are not trying to memorize Git syntax.

You are learning to:

- define tasks clearly
- supervise execution
- verify results

That is the new workflow.

Not manual repetition.
Directed automation.

---

## What to Watch For

Even with Claude, you still need judgment.

Check:

- the remote URL is correct
- the branch is `main`
- the push succeeded
- the repo contains the expected files

Claude accelerates execution.
You validate the result.

---

## Builder Principle

Do not use Claude to avoid understanding.

Use Claude to avoid friction.

You still need to know:

- what a repository is
- what a remote origin is
- why pushing matters

But you do not need to perform every step manually.

---

## Takeaway

With Claude Code, Git becomes a conversation.

You don’t type commands from memory.

You say what you want:

> initialize the repo
> connect GitHub
> push the project

Claude handles the mechanics.

You keep building.

**Next chapters willinclude:**

- Building your first real website
- Styling with modern CSS
- Using React / Next.js
- Deployment (live website)
- SEO & performance

---

# Chapter 3 — Build Your First Website with Claude Code

## 3.1 From Idea to Website

Now that you:
understand the terminal
have Claude Code installed

You're ready to build your **first real website**.

The key shift here is simple:

☑ You don't code everything manually
☑ You *direct Claude like a product manager*

---

## 3.2 Define What You Want

Before typing anything, be clear.

Bad prompt:

```
Create a website
```

Good prompt:

```
Create a modern personal website with:
- a homepage
- an about section
```

```
- a contact form
- responsive design
```

The better your instructions, the better the result.

---

## 3.3 Initialize Your Project

In your terminal:

```
mkdir my-first-site
cd my-first-site
claude init
```

Claude will prepare the project environment.

---

## 3.4 Generate Your Website

Now ask Claude:

```
Create a clean landing page for a developer portfolio using HTML, CSS
and a bit of JavaScript
```

Claude will typically:

- create files (`index.html`, `style.css`, etc.)
- structure folders
- explain choices

---

## 3.5 Understand What Claude Generated

Don't skip this step.

Open your project:

```
code .
```

You'll usually see:

```
/my-first-site
   ├─── index.html
   ├─── style.css
```

```
├── script.js
```

**index.html**

Structure of your site

**style.css**

Design (colors, layout, spacing)

**script.js**

Interactions (buttons, animations)

---

## 3.6 Run Your Website

If Claude used a simple setup:

☑ Just open `index.html` in your browser

If it created a modern project (React / Next.js):

```
npm install
npm run dev
```

Then go to:
http://localhost:3000

---

## 3.7 Iterate with Claude (This is the Magic)

Your first version won't be perfect — and that's normal.

Now you refine.

Examples:

```
Make the design more modern and minimal
Add a dark mode toggle
Improve mobile responsiveness
Add smooth scrolling animations
```

 You're not coding line-by-line
 You're *guiding evolution*

---

## 3.8 Fixing Issues with Claude

If something breaks:

Instead of debugging alone, ask:

```
This button is not working, can you fix it?
```

Or even better:

```
Explain why this JavaScript function is not triggering
```

Claude will:

- identify the issue
- suggest fixes
- explain the logic

---

## 3.9 The Builder Mindset

At this stage, your role is:

Not → developer
But → **builder / director**

You:

- define vision
- validate results
- iterate quickly

Claude:

- executes
- suggests improvements
- accelerates everything

---

## 3.10 Common Beginner Mistakes

**Being too vague**

→ Claude gives generic results

**Not reviewing code**

→ You don't learn

**Asking everything at once**

→ Hard to control output

---

## 3.11 Pro Tips

Work step-by-step
Ask for explanations
Keep prompts simple but precise
Save versions of your project

---

## 3.12 Your First Milestone

At this point, you should have:

a working website
a basic structure
confidence using Claude

That's already more than most beginners.

---

## End of Chapter 3

Next step □
□ Make your website **look professional**

---

# Next Chapter Preview

**Chapter 4 — Design Like a Pro (Without Being a Designer)**

You'll learn:

- modern UI principles
- how to make your site look clean instantly
- how to use Claude to upgrade design

---

Perfect. Now we apply **true Impeccable.style principles** — not just tone, but **method + vocabulary + constraints**.

---

# Chapter 5 — Design with Impeccable (Make AI Output Look Professional)

## 5.1 The Real Problem

AI doesn't fail at coding.
It fails at taste.

Default output looks like:

- gradient backgrounds
- centered layouts
- cards inside cards
- generic fonts

This is not random.

It's what happens when you **don't specify design intent**. (Emelia)

---

## 5.2 Impeccable Is a Language

Impeccable is not a tool.
It's a **vocabulary layer between you and AI**. (Emelia)

Instead of:

> "make it better"

You say:

- `/arrange`
- `/typeset`
- `/distill`
- `/polish`

You stop guessing.
You start directing.

---

## 5.3 Rule #1 — Distill (Remove Before Adding)

Bad design = too much.

Impeccable principle:
☑ Remove everything that doesn't earn its place

```
/distill
```

Result:

- less noise
- stronger message
- faster understanding

Simplicity is not minimalism.
It's **precision**.

---

## 5.4 Rule #2 — Visual Rhythm (Spacing Is Not Uniform)

Most beginners do this:

same padding everywhere
same spacing everywhere

Result:
→ flat, lifeless UI

Impeccable:

☑ vary spacing
☑ create rhythm

- tight groups → related content
- large gaps → separation

Without rhythm, layouts feel monotonous. (Mintlify)

```
/arrange
```

---

## 5.5 Rule #3 — Kill the Card Addiction

Default AI output:

```
card
  card
    card
```

This is a known anti-pattern. (AgentSkills)

Impeccable rule:

□ Not everything needs a container
□ Use spacing, not boxes

Cleaner. Lighter. More premium.

---

## 5.6 Rule #4 — Asymmetry Over Boring Grids

Centered layouts = safe
Safe = forgettable

Impeccable:

□ left-aligned text
□ asymmetric layout
□ controlled imbalance

Asymmetry feels designed.
Symmetry feels templated. (Mintlify)

---

## 5.7 Rule #5 — Color Discipline

Default AI colors:

- neon
- gradients
- random accents

Impeccable:

☑ 1 dominant color
☑ 1 accent
☑ tinted neutrals

Never:

- pure black (#000)
- pure white (#fff)

Always tint them. (AgentSkills)

`/colorize`

---

## 5.8 Rule #6 — Typography = Authority

Bad:

- Inter everywhere
- random sizes
- weak hierarchy

Impeccable:

☑ strong type scale

☑ clear hierarchy

☑ intentional pairing

Typography is not decoration.
It's structure.

`/typeset`

---

## 5.9 Rule #7 — Stop Centering Everything

Centered layout = default AI behavior

Impeccable:

☑ align left

☑ create flow

☑ guide the eye

Design is movement.

Users scan.
You control the path.

---

## 5.10 Rule #8 — Motion Is Rare, Not Constant

Bad:

- animations everywhere
- bouncing effects
- gimmicks

Impeccable:

☑ one strong moment

☑ purposeful motion

Example:

- page load animation

- smooth reveal

Not:

- every button dancing

Motion should **add meaning**, not noise. (Mintlify)

---

## 5.11 The Impeccable Workflow

You don't ask once.
You **chain commands**.

Example:

```
/arrange
/typeset
/colorize
/distill
/polish
```

Each pass:

- removes friction
- improves clarity
- increases perceived quality

This is how you go from:

"AI-generated"

to

"designed"

---

## 5.12 Anti-Patterns You Must Avoid

Never let Claude do this:

gray text on colored background
nested cards
identical feature grids
gradient overload

default fonts
centered everything

These are known failure patterns. (AgentSkills)

---

## 5.13 The Impeccable Standard

A good UI:

- works

An impeccable UI:

- feels obvious
- feels intentional
- feels expensive

Every element:
□ justified
□ placed with intent
□ contributing to clarity

---

## End of Chapter 5

You now have:
a design vocabulary
a refinement process
control over AI output

---

# Next Chapter

**Chapter 6 — Deploy Like a Builder (Put Your Website Online)**

Because:
☑ if it's not live, it doesn't exist.

---

Continuing in **Impeccable.style** — direct, practical, zero fluff.

---

# Chapter 6 — Deploy with GitHub Pages (Make It Live)

## 6.1 Reality Check

If your website is not online:
☑ it does not exist

Localhost is not a product.
Deployment is the moment it becomes real.

---

## 6.2 What Is GitHub Pages

GitHub Pages is simple:

- free hosting
- instant deployment
- tied to your GitHub repo

No server.
No DevOps.
No excuses.

---

## 6.3 Prerequisite

You need:

- a GitHub account
- your project folder ready

That’s it.

---

## 6.4 Initialize Git (One Time Setup)

Inside your project:

```
git init
git add .
git commit -m "initial commit"
```

You just created a versioned project.

---

## 6.5 Create a Repository

Go to GitHub.

Click:
☑ **New Repository**

Name it:

```
my-website
```

Then copy the repo URL.

---

## 6.6 Connect Your Project to GitHub

Back in your terminal:

```
git remote add origin https://github.com/your-username/my-website.git
git branch -M main
git push -u origin main
```

Your code is now online.

Not live yet — but close.

---

## 6.7 Activate GitHub Pages

On GitHub:

1. Go to your repository
2. Click **Settings**
3. Scroll to **Pages**

Then:

- Source → `main` branch
- Folder → `/ (root)`

Click **Save**

---

## 6.8 Your Website Is Live

GitHub gives you a URL:

```
https://your-username.github.io/my-website/
```

Open it.

> Your website is now public

---

## 6.9 Update Workflow (The Only Loop You Need)

Every change:

```
git add .
git commit -m "update"
git push
```

Wait a few seconds.
Refresh the page.

That's deployment.

---

## 6.10 Common Mistakes

> **Nothing shows**

→ Check:

- file is named `index.html`
- it's in the root folder

---

### Styles not loading

→ Paths are wrong

Use:

```
<link rel="stylesheet" href="./style.css">
```

Not:

```
<link rel="stylesheet" href="/style.css">
```

---

### Page not updating

→ Cache issue

Hard refresh:

- Mac: `Cmd + Shift + R`
- Windows: `Ctrl + Shift + R`

---

## 6.11 Custom Domain (Optional, Powerful)

Instead of:

```
your-username.github.io
```

You can use:

```
yourname.com
```

Steps:

1. Buy a domain (e.g. Namecheap or GoDaddy)
2. In GitHub Pages → add your domain
3. Configure DNS (A records)

Now your site is:
☑ personal

☑ credible

☑ real

---

## 6.12 Claude Workflow for Deployment

You don't guess.

You ask:

```
Prepare this project for GitHub Pages deployment
Fix all asset paths for static hosting
Optimize for fast loading
```

Claude removes friction.

---

## 6.13 The Builder Standard

At this point, you have:

- a website
- a design system
- a live URL

You are no longer learning.
You are shipping.

---

## End of Chapter 6

Next step ☑

# Chapter 7 — Make It Fast, Visible, and Useful

Because a live website is step one.
A successful website is something else.

Continuing in **Impeccable.style** — sharp, practical, outcome-driven.

---

# Chapter 7 — Make It Fast, Visible, and Useful

## 7.1 The Truth

A live website is not success.

If no one sees it:
☑ it doesn't exist

If it's slow:
☑ people leave

If it's unclear:
☑ people don't act

---

## 7.2 Speed Is a Feature

Users don't wait.

You have:
☑ ~3 seconds

After that → they're gone.

---

## 7.3 The 80/20 of Performance

You don't need perfection.
You need obvious wins.

### 1. Compress Images

Big images = slow site

Use:

- `.webp` format
- small dimensions

Ask Claude:

```
Convert all images to optimized webp format and reduce size
```

---

### 2. Remove What You Don't Need

Unused code = slower load

```
Remove unused CSS and JavaScript
```

---

### 3. Keep It Static

GitHub Pages is fast because:

no server
no database

Don't overcomplicate.

---

## 7.4 Measure Speed

Use:

Google Lighthouse

Open Chrome → DevTools → Lighthouse → Run audit

Focus on:

- Performance score
- First Contentful Paint
- Largest Contentful Paint

---

## 7.5 Clarity Beats Cleverness

Most websites fail here.

Bad:

- vague headlines

- buzzwords
- no direction

Good:

☑ clear message
☑ clear audience
☑ clear action

Test:

Can someone understand your site in **5 seconds**?

If not → fix it.

---

## 7.6 The One Action Rule

Your site should push ONE action:

- contact you
- buy something
- subscribe

Not 5 options.
Not confusion.

Tell Claude:

```
Improve call-to-action clarity and focus on one primary action
```

---

## 7.7 SEO Basics (Simple, Enough)

You don't need advanced SEO.

You need basics done right.

### 1. Title Tag

```
<title>Build Websites with AI | Your Name</title>
```

---

### 2. Meta Description

```
<meta name="description" content="Learn how to build modern websites using Claude Code.">
```

---

### 3. Headings Structure

```
<h1>Main title</h1>
<h2>Sections</h2>
<h3>Details</h3>
```

---

### 4. Clean URLs

Not:

```
/page?id=123
```

But:

```
/portfolio
```

---

## 7.8 Content Wins Long-Term

Design attracts.
Content converts.

Add:

- a clear value proposition
- real examples
- simple explanations

No fluff.
No filler.

---

## 7.9 Analytics (Know What Happens)

Install:

Google Analytics

Track:

- visitors
- pages viewed
- behavior

Without data:
☑ you guess

With data:
☑ you improve

---

## 7.10 Iterate Like a Builder

Loop:

1. Launch
2. Observe
3. Improve

Ask Claude:

```
Suggest improvements to increase conversions
Analyze this landing page and make it clearer
```

---

## 7.11 The Impeccable Standard

A strong website is:

- fast
- clear
- focused

Not:

- complex
- flashy
- overloaded

---

## 7.12 Final Reality

Most people:

- overbuild
- overdesign
- never launch

You did:

build
design
deploy
optimize

That's rare.

---

## End of Chapter 7

---

Got it. Clean, sharp conclusion — aligned with **Impeccable.style**.

---

# Conclusion — You Already Have What You Need

## The Illusion

You think you need:

- more tutorials
- more tools
- more knowledge

You don't.

You needed:
☑ a process
☑ a way to ship

Now you have both.

---

## What Changed

Before:

- you consumed
- you hesitated
- you searched

Now:

- you build
- you decide
- you iterate

That's the shift.

---

## What You Can Do Now

You can:

- launch a personal website
- build landing pages in hours
- create projects without friction
- work with AI instead of against it

This is not theoretical.

It's executable. Today.

---

## The Advantage

Most people:

- overthink
- overdesign
- never publish

You:

- shipped
- improved
- learned by doing

That alone puts you ahead.

---

## The Only Rule

Don’t stop.

Not:

- after one website
- after one success
- after one failure

Keep the loop:

☑ build → launch → refine

---

## Final Principle

Clarity > complexity
Speed > perfection
Action > knowledge

---

## Last Line

You don’t need to become a developer.

You already became a builder.

Now:

**Build something real.**

# The End